comfy_agent

How the node works

A ComfyUI meta-node that builds workflows from a natural-language prompt, edits the workflow already on your canvas, and answers questions about your specific installation - entirely on your own machine, using a local Ollama model.

Modes	BUILD, MODIFY, CHAT
Runs on	ComfyUI V3 node API + local Ollama
Network	localhost only by default; nothing leaves the machine
Verified against	ComfyUI v0.32.0, 1767 installed node types

Generated from the source at docs/make_pdf.py.
Every figure in this document was measured, not estimated.

1. What the node is

and the problem it solves

ComfyUI workflows are graphs. Building one means knowing which of ~1,700 installed node types you need, what each one's inputs and outputs are called, and how they connect. That knowledge is the barrier for new users and tedious for everyone else.

comfy_agent is a node you drop on the canvas. You type what you want in plain language; it writes the workflow around itself. It is a meta-node: it does not process images or latents. It reads the ComfyUI registry, talks to a local language model, and draws on the canvas.

Three modes

Mode	What it does	Needs a model?
BUILD	Creates a new workflow from a prompt. Tries a pre-validated blueprint first; falls back to generating one.	Only for free-form
MODIFY	Edits the workflow already on your canvas - add, remove, or change widget values. Fully undoable.	Yes
CHAT	Answers questions about ComfyUI, your installed nodes and models, your current graph, and your last error.	Yes

What makes it different from a chatbot in a browser tab

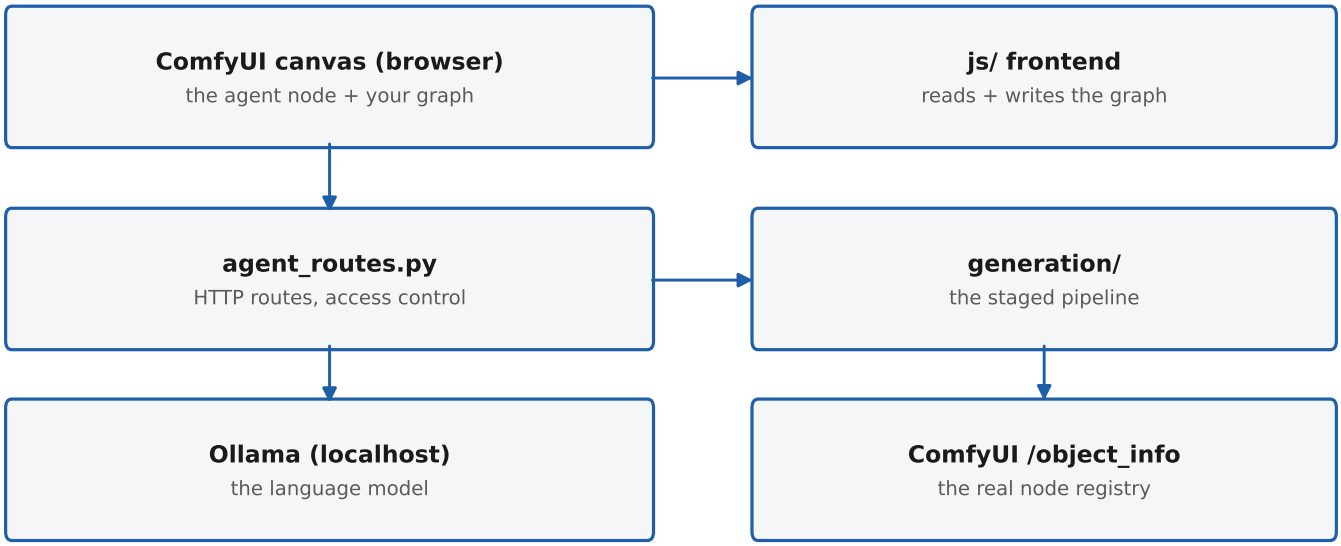
- It reads your actual installation. Node suggestions come from the ~1,700 node types you really have, not from what the model remembers about ComfyUI.
- It reads your actual canvas. CHAT and MODIFY both receive the current graph, so "why is this broken?" is a question about your workflow.
- It writes to the canvas directly. There is no copy-pasting JSON.
- It runs locally. No prompt, workflow, or file listing leaves the machine.

> **The core design constraint**
>
> The default model is small - under a billion parameters - so it fits alongside image generation on one GPU. Almost every design decision in this node exists to make a small model reliable: narrow questions, filtered context, and validating every answer against the real registry rather than trusting it.

2. How the pieces fit together

three layers, one round trip

The node is split across a Python backend, a JavaScript frontend, and the local Ollama server. The frontend owns the canvas; the backend owns generation. They meet at one HTTP request and a WebSocket for progress.



What each layer is responsible for

Layer	Responsibility
js/canvas_writer.js	Creating nodes, wiring links, finding empty canvas space, snapshotting for undo, applying edits.
js/agent_node_ui.js	The node's buttons and status line; sends the request; reports what happened.
agent_routes.py	Four HTTP routes, localhost-only access control, concurrency limits.
generation/	The staged pipeline that turns a prompt into a validated graph.
knowledge/	Filtering the registry, validating link types, matching blueprints.
install_index.py	A searchable index of every installed node, for CHAT.

Why the backend never touches the canvas

ComfyUI's graph lives in the browser. The backend only ever returns a description of what to draw; the frontend decides where it fits and applies it. That split is why an agent action can be snapshotted and undone.

3. BUILD mode

two paths, and why the fast one exists

When you press Build Workflow, the node takes one of two paths. The choice is made before any model is consulted.

Path 1 - blueprint (instant, no model)

Eight complete workflows ship with the node, each validated against a real ComfyUI install. Your prompt is scored against their trigger keywords; a match is instantiated directly. This takes milliseconds, cannot hallucinate, and works with Ollama stopped.

```
txt2img_sdxl     txt2img_flux     img2img_base     controlnet_base
lora_stack       face_detailer    upscale_latent   video_wan
```

Model-family keywords carry extra weight, so "flux txt2img" selects the FLUX blueprint rather than the SDXL one.

Path 2 - free-form (the staged pipeline)

If nothing matches, the node generates a workflow from scratch. This is where the small-model problem is real, and it is covered on the next page.

What you get back

Outcome	What the node does
Complete workflow	Places it, pans the canvas to it, reports node and link counts.
Incomplete workflow	Still places it. Nodes with unconnected required inputs are coloured amber and titled with what they need. The status says it cannot run yet.
Failure	Nothing is drawn; the status names the cause.

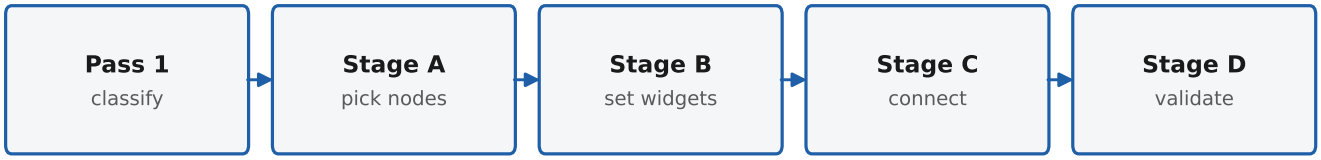
An honest partial result beats a false success

A graph missing one connection is still worth having - you can finish it by hand in seconds. What is not acceptable is calling it ready and letting you discover the gap when the queue fails. The node distinguishes 'built' from 'runnable' everywhere.

4. The generation pipeline

why one big question becomes five small ones

Asking a small model to emit a whole workflow in one go produces confident nonsense. Instead the work is split into stages, each asking for one narrow thing, and each output is checked against the real registry before the next stage runs.



Stage	Asks the model	Then checks
Pass 1	Which categories does this task touch?	Core categories are always added, so a bad guess cannot drop the loaders.
Stage A	Which node types are needed?	Every name must exist in the registry. Invented names are dropped; a suspiciously short list triggers a retry on a bigger model.
Stage B	What values for this one node's widgets?	Types coerced, numbers clamped to the declared range, dropdown values checked against the real options. One call per node, run in parallel.
Stage C	Which outputs connect to which inputs?	Every link re-validated for type compatibility. Unfilled required inputs trigger a targeted repair pass that asks only about the gaps.
Stage D	(nothing - pure code)	Structural check: no orphans, no empty spec, and whether the graph can actually run.

Keeping the context small

The full registry is about 2.7 MB - far more than a small model can hold. Pass 1's categories filter it, and a character budget trims what remains to roughly 2,500 tokens. Essential nodes such as the checkpoint loader and image saver are admitted before the budget applies, so they can never be squeezed out by a large custom pack.

5. MODIFY mode

editing a workflow you already have

MODIFY changes what is on your canvas instead of building something new. It does not regenerate the workflow - it produces a diff, listing only what should change.

```
{ "add_nodes":       ["PreviewImage"],
  "remove_node_ids": [7],
  "set_widgets":     [{"node_id": 5, "widget": "steps", "value": 35}] }
```

A diff rather than a rewrite for two reasons. You asked to change one thing, so re-rolling everything would discard the parts you were happy with. And a diff can be shown to you and reversed; a wholesale replacement cannot.

Everything is checked before anything is applied

- Node ids must exist on your canvas. An invented id is rejected, not deleted.
- Added node types must be installed here.
- Widget names must exist on the node being changed.
- Values must be scalars, then type-coerced and range-clamped - the same checks BUILD applies.
- A diff that would delete most of the canvas triggers a second opinion from a larger model before it is trusted.

Undo

Before any agent action writes to the canvas, the whole graph is snapshotted. Undo Last Agent Action restores it. Undo is what makes the feature usable at all: without it, one bad edit teaches you never to press the button again.

The snapshot restores the entire canvas to that moment, so manual edits made after the agent ran are reverted too.

MODIFY needs a capable model

Reasoning about node ids that already exist is the hardest thing this node asks of a model. Measured on a real 7-node canvas: qwen3.5:4b and :9b produced the correct minimal edit every time, while the sub-1B model edited the wrong node with the wrong value. MODIFY therefore runs on the escalation model by design, and warns you on the node if none is installed. Run: ollama pull qwen3.5:4b

6. CHAT mode

questions about your machine, not about ComfyUI in general

Selecting CHAT swaps the Build button for a conversation panel inside the node. What makes it useful is what it can see. Every question is answered with four things in context:

- Your current canvas - trimmed to node types, titles, widget values and links.
- The installed nodes most relevant to your question, retrieved from an index of every node type on this machine.
- Your actual model files - checkpoints, LoRAs, VAEs, upscalers.
- Your most recent execution error, captured automatically. You never paste a traceback.

How retrieval works

The registry is far too large to put in a prompt, so every node is described in one line and embedded once, then cached. Each question retrieves the top matches - typically 24 nodes - and only those are shown to the model.

Three ranking details matter more than they look. Node names are split on case boundaries, so "FaceDetailer" can match "make faces look better". Filler words are excluded from name matching, because "how do I make..." otherwise ranks a node called MakeTrainingDataset first. And naming a node exactly outranks its variants, so asking about KSampler does not return KSamplerAdvanced instead.

The index

Property	Measured on a 1,767-node install
First build	About 21 seconds, once. Starts in the background at ComfyUI startup.
Later starts	Loaded from cache, about 1 second.
After installing a node pack	Only the new nodes are embedded - about 1 second, not a full rebuild.
If the embedding model is missing	Falls back to keyword search. Chat still works.

CHAT replaced three planned modes

EXPLAIN, DEBUG and SUGGEST were originally separate. In practice the distinction collapsed: someone asking "why is my image black?" wants an explanation, a diagnosis and a suggestion in one answer. Making them choose a mode first was friction with no payoff, so CHAT covers all three.

7. Safety, privacy and failure

what the node will and will not do

Privacy

Everything runs on your machine. The node talks to ComfyUI on localhost and Ollama on localhost. Your prompts, your workflows, and the list of your model files are never sent anywhere else. There are no API keys and no accounts.

Access control

All routes reject non-localhost callers by default. Remote access is opt-in per subnet and can require a shared token. Cross-origin requests are refused, so a web page you happen to have open cannot drive your canvas or burn your GPU.

How it fails

When	What happens
Ollama is stopped	Blueprint builds still work. The node says so rather than appearing broken.
A model is missing	The node reports it on creation, with the exact ollama pull command.
The model invents a node	The name is not in the registry, so it is dropped before anything is drawn.
A link cannot be wired	The build still lands; the status names each link that was skipped and why.
A built workflow errors	The node notices the failure came from nodes it just placed and says so, keeping the error for CHAT.
Anything else	The build fails with a stated reason. The canvas is left alone.

The principle behind all of it

A wrong workflow that claims to be right costs far more than a refusal. Every stage validates against the real registry rather than trusting the model, and anything that cannot be verified is reported rather than hidden.

8. Running it

setup, models, and where to look when something breaks

Requirements

ComfyUI with the V3 node API, and Ollama running locally.

```
ollama pull qwen3.5:0.8b      # base model - free-form builds
ollama pull qwen3.5:4b        # escalation - required for MODIFY
ollama pull nomic-embed-text  # semantic search for CHAT
```

Blueprint builds need none of these. Free-form builds work on the base model alone. MODIFY and answer quality both improve materially with the 4B.

Check the setup before blaming the node

The node calls a health endpoint when it is created and shows the first real problem in its own status line, with the command that fixes it. For the full picture:

```
curl http://127.0.0.1:8188/agent/health
```

That reports whether Ollama is reachable, which models are present, whether the search index is built, and how many blueprints loaded.

Start here

The examples/ folder holds four loadable workflows, one per mode. The first uses a blueprint and works with Ollama stopped - which makes it a clean way to tell an installation problem apart from a model problem.

Tuning

If	Change
Image generation runs out of memory while chatting	Lower the chat context window in config.py - retrieval means answers barely change.
Free-form builds wire poorly	Pull a larger escalation model.
Builds are too large or too small	Adjust max_nodes on the node itself (4-20).
You want a new instant workflow	Save a working graph into blueprint_defs/ and add its trigger keywords.

Reference material lives beside this document: README.md for setup and troubleshooting, ARCHITECTURE.md for the design record, and examples/README.md for the sample workflows.